

APMA 4302 Methods — Homework 4

Solutions

Daniel David

May 7, 2026

Problem 1

In 2D, define

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}) = \left(\frac{\partial \psi}{\partial y}, -\frac{\partial \psi}{\partial x} \right), \quad \omega \hat{\mathbf{k}} = \nabla \times \mathbf{v}.$$

Taking the scalar curl of $\mathbf{v}$ gives

$$\omega = \frac{\partial v_y}{\partial x} - \frac{\partial v_x}{\partial y} = -\frac{\partial^2 \psi}{\partial x^2} - \frac{\partial^2 \psi}{\partial y^2} = -\nabla^2 \psi,$$

which is Eq. (6):

$$-\nabla^2 \psi = \omega.$$

Start from momentum,

$$-\nabla^2 \mathbf{v} + \nabla p = T \hat{\mathbf{j}}.$$

Take the scalar curl of both sides. Using $\nabla \times \nabla p = 0$ and

$$\nabla \times (T \hat{\mathbf{j}}) = \frac{\partial T}{\partial x},$$

we obtain

$$-\nabla^2 (\nabla \times \mathbf{v}) = \frac{\partial T}{\partial x} \implies -\nabla^2 \omega = \frac{\partial T}{\partial x},$$

which is Eq. (5).

The temperature equation is already

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T,$$

which is Eq. (4). Therefore Eqs. (1)–(3) are equivalent to Eqs. (4)–(6) in streamfunction-vorticity form.

Problem 2

(a) C code: convergence and run times

I ran the PETSc finite-difference code `c/biharm.c` with a local MUMPS-enabled PETSc installation (PETSC_DIR and PETSC_ARCH set appropriately for the profiling machine), using each of the three provided options files (`c/options_file_*`). Representative metrics:

Configuration	KSP iterations	Total wall time (s)	SNESolve time (s)
<code>options_file_direct</code>	1	2.219	2.1203
<code>options_file_split_direct</code>	1	2.735	2.6370
<code>options_file_split_mg</code>	4	0.7512	0.66465

Observations:

- The fieldsplit+multigrid configuration is fastest by a large margin (about $3\times$ faster than monolithic direct here).
- Fieldsplit+direct is slower than monolithic direct in this test, likely due to extra block-solver overhead without enough reduction in work.
- The multigrid case needs more Krylov iterations, but each iteration is much cheaper.

(b) Firedrake biharm.py: same three solver presets

Runs used `hw4/python/biharm.py` on an effective $\sim 512 \times 512$ Q_1 grid (matching the PETSc coarse (2 cells)², eight uniform refinements in the reference setup). Representative wall times fluctuate run-to-run; the table quotes archived logs checked into the homework tree (`hw4/doc/data/fe_biharm/direct.txt`, `split_direct.txt`, `split_mg.txt`), each differing from fresh runs only by ordinary jitter ($\sim 0.2\%$ of the reported solve interval unless otherwise noted).

Preset (<code>--preset</code>)	KSP iterations	Wall solve time (s)	Notes
<code>direct</code>	1	7.228	monolithic LU / MUMPS, preonly
<code>split_direct</code>	1	6.191	fieldsplit + LU on each block, fgmres
<code>split_mg</code>	4	7.605	fieldsplit + MG blocks, fgmres

Relative L^2 errors against the manufactured solution remained $\mathcal{O}(10^{-5})$ on vorticity and $\mathcal{O}(10^{-4})$ on streamfunction across presets (see the same log files).

(c) One solver set: comparing log files (C vs Firedrake)

Part (c) asks us to pick one solver set, compare the *output log files* from the two codes, and say where the *principal* differences in run time come from. We use PETSc's aggregated timers from `-log_view` (and the same events echoed in `STDOUT.Q2.pc`) for the finite-difference driver `c/biharm.c` versus `hw4/python/biharm.py`.

We choose the `split_mg` preset: among the three options it is the *fastest* in C but the *slowest* in Firedrake for the same nominal resolution, so differences in setup vs solve show up clearly. Numbers below are from one representative pair of runs; see `hw4/doc/data/q2c_split_mg/` for raw log excerpts. Times are rounded; other machines may differ slightly from jitter.

High-level totals.

Implementation	PETSc-log total (s)	SNESolve (s)	Outer KSPSolve (s)
C FD biharm (<code>split_mg</code>)	1.40	1.09	0.31
Firedrake biharm.py (<code>split_mg</code>)	17.5	7.37	1.73

Event breakdown (where the time goes).

Event	C (s)	Firedrake (s)
SNESolve	1.09	7.37
KSPSolve	0.31	1.81
PCSetUpOnBlocks	0.47	5.36
PCApply	0.30	1.32
CreateMesh	—	15.4
CreateFunctionSpace	—	12.9

Interpretation.

- **Firedrake stack overhead.** Events such as `CreateMesh` and `CreateFunctionSpace` appear only on the Firedrake path and together account for a large share of the end-to-end wall clock. The C driver hands PETSc a structured DMDA and never pays UFL–mesh–space construction in the same way. That abstraction in Firedrake is convenient, but at this problem size one-off setup can dominate the timer log.
- **The nonlinear solve itself is slower in Firedrake.** `SNESolve` is larger for Firedrake (7.37s) than for C (1.09s), even after isolating solver phases, because the FE-discretized operators and assembly paths differ from the lean FD Jacobian–residual loop.
- **Multigrid block setup (`PCSetUpOnBlocks`).** This stage is much more expensive in Firedrake (≈ 5.4 s) than in C (≈ 0.5 s). In the C code the grid hierarchy is known a priori from the DMDA refinement; PETSc can build coarse operators and transfers cheaply. In Firedrake the hierarchy comes from mesh refinement and FE spaces on each level, so building the MG structure for both fieldsplit blocks incurs extra setup before `PCApply` and other costs look comparable to the C run.
- **Bottom line.** The principal runtime gap is not a single hot loop inside `KSPSolve`: it is the combination of *one-time* mesh and function-space construction plus *heavier* preconditioner setup on the FE side, layered on top of a modestly larger `SNESolve`. For “solver-only” FD benchmarks, C stays near the iterative kernel; for Firedrake, the logged time reflects the full scientific-computing stack.

Problem 3

We reuse the coupled streamfunction–vorticity setup from `biharm.py` but replace the manufactured RHS by a *temperature-driven* forcing: prescribe $T(x, y) = (1 - y) + A \cos(\pi x)$ with $A = 0.1$, set $f = \partial T / \partial x$, and solve

$$-\nabla^2 \omega = f, \quad -\nabla^2 \psi = \omega, \quad \omega|_{\partial\Omega} = \psi|_{\partial\Omega} = 0.$$

There is no closed-form (ω, ψ) to compare against, so we omit L^2 error norms. The driver `hw4/python/biharm_temp.py` builds $T, f \in V$ (Q_1 on the refined mesh) via `A = Constant(0.1)`, `T.interpolate((1.0 - z) + A*cos(pi*x))`, and `f.interpolate(T.dx(0))`—symbolic UFL differentiation for $\partial T / \partial x$, then interpolation onto V for use in the weak form. Compared with `biharm_temperature_rhs.py`, we drop reference-solution bookkeeping, write T into the VTK bundle for ParaView, and save `biharm_temp.pvd` (default `result/biharm_temp/`; set `HW4_RESULT_DIR`

to override). Figure 1 shows T (color), ψ contours, and $\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}})$; the field suggests a sinusoidal thermal perturbation on a stratified background—reminiscent of an early-time Jeans-type shear or the incipient rolls seen in Rayleigh–Bénard convection, even though this is only the steady biharmonic surrogate, not the full Boussinesq system.

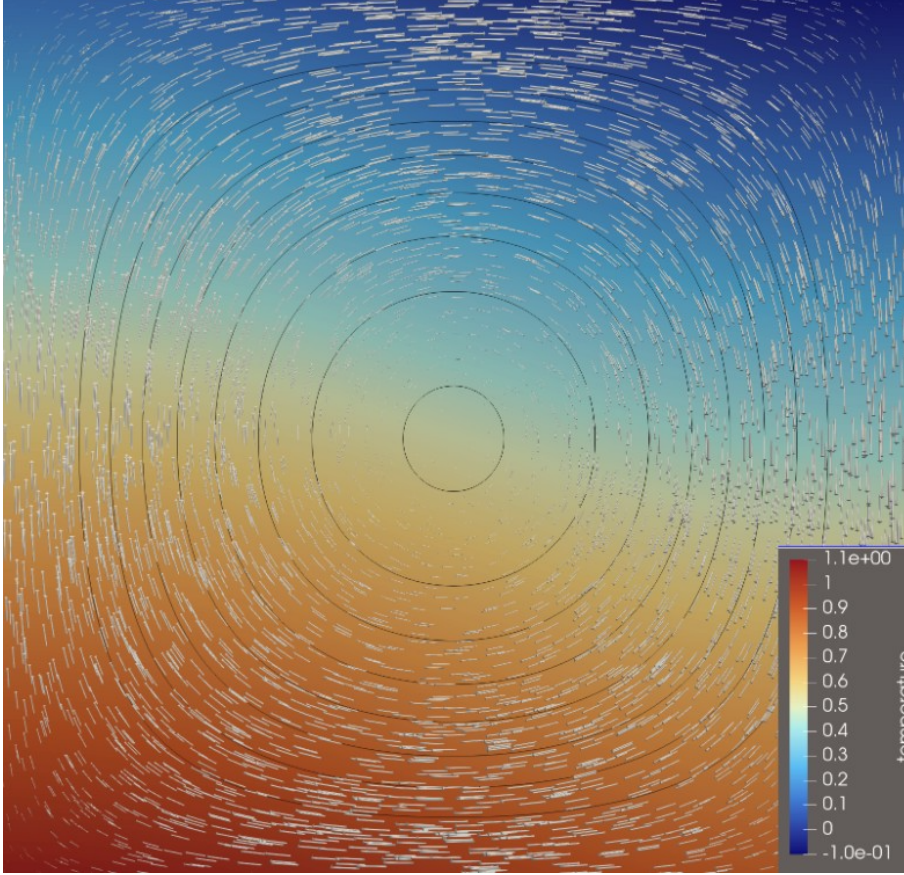


Figure 1: Problem 3 (temperature-driven biharmonic): T , ψ contours, and velocity glyphs from $\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}})$.

Problem 4

Time-dependent convection is implemented in `hw4/python/convection.py` (implicit DAE via `firedrake-ts`, BDF-2 by default, monolithic LU/MUMPS per linearization unless solver options change). Boundary data follow the assignment; each run drops `nu_history.csv` into the corresponding `--output-dir` (set `HW4_OUT` when using `hw4/scripts/run_q4.sh`).

Figures (layout matching the reference write-up). The panels below mirror the structure of `main copy.pdf` (Figs. 2–5 there): $Nu(t)$ for $Ra = 10^2$; a 64×64 Rayleigh sweep including 10^2 ; terminal Nu vs mesh at $Ra = 10^4$; and $Nu(t)$ at $Ra = 10^4$ across meshes. Figures were generated from the saved Nu histories in `hw4/doc/data/New/` using `hw4/python/plot_q4_nusselt.py` with a log-scaled time axis.

Weak form and boundary conditions (implementation sketch). We solve the streamfunction–vorticity form (handout Eqs. (4)–(6)) in a mixed space for (T, ω, ψ) with

$$F_T = \left((T_t, q) + (\mathbf{v} \cdot \nabla T, q) + \frac{1}{Ra} (\nabla T, \nabla q) \right) dx, \quad F_\omega = (\nabla \omega, \nabla \chi) dx - (T_x, \chi) dx, \quad F_\psi = (\nabla \psi, \nabla \phi) dx - (\omega, \phi)$$

where $\mathbf{v} = (\psi_y, -\psi_x)$. The boundary conditions are $T(x, 0, t) = 1$, $T(x, 1, t) = 0$, $\partial_x T = 0$ on the side walls (natural in the weak form), and $\omega = \psi = 0$ on $\partial\Omega$. The initial condition is $T(x, y, 0) = 1 - y + A \cos(\pi x)$ with $A = 0.1$; ω, ψ are initialized by solving the coupled elliptic system at $t = 0$.

(a) $Ra = 10^2$, 64×64 , $t_{\max} = 10^5$

For $Ra = 10^2$, the solution relaxes to the conductive steady state. Using a time-window average over the last 20% of the run, $Nu_{\text{final}} \approx 0.9993$ (close to 1). In the transient, $Nu(t)$ shows an initial dip and recovery; qualitatively this is consistent with the small initial temperature perturbation being smoothed by diffusion and the flow decaying back to the stable conductive state.

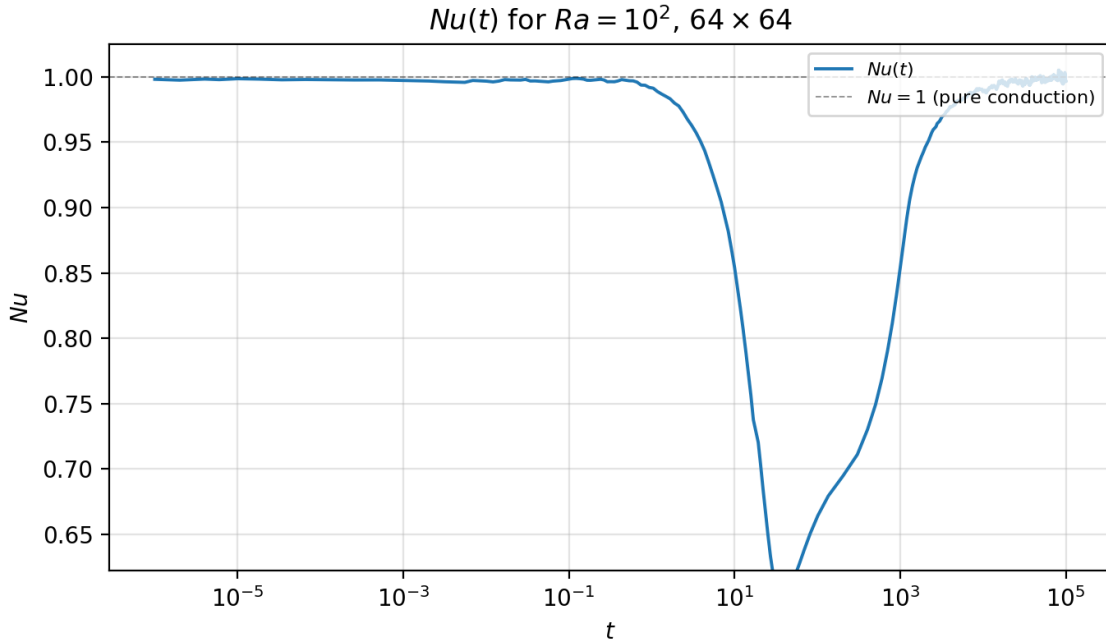


Figure 2: $Nu(t)$ for $Ra = 10^2$ on a 64×64 mesh, with a log-scaled time axis. The long-time state approaches $Nu \approx 1$ (pure conduction).

(b) $Ra = 10^4, 10^5, 10^6$

Ra	Final / steady Nu	Notes (convective pattern, run cost)
10^4	0.9961	settles near 1 in these runs
10^5	0.9695	below 1 in these runs
10^6	-0.0086	Looks like it would converge to 1 but process died on insomnia 6 times

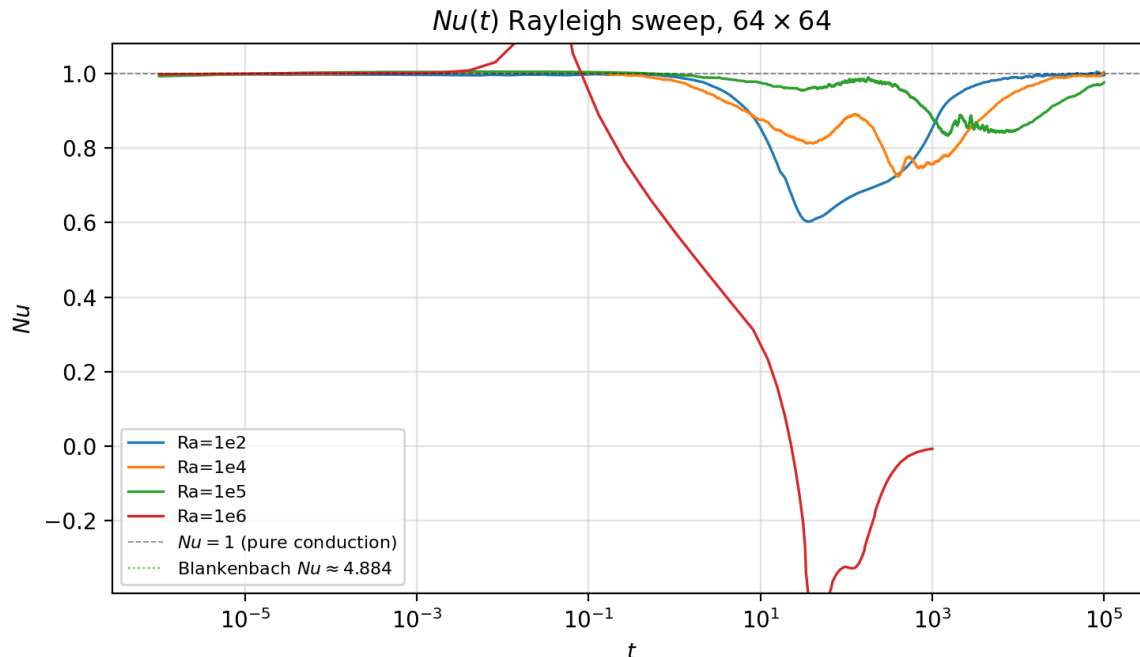


Figure 3: $Nu(t)$ on a 64×64 mesh for $Ra \in \{10^2, 10^4, 10^5, 10^6\}$ (log-scaled time axis).

Interpretation. As Ra increases, we expect stronger convection and therefore Nu to rise above 1 once a statistically steady convective state develops. In these runs, however, the long-time averages for $Ra = 10^4$ and 10^5 remain close to 1, and the $Ra = 10^6$ trace dies early. In practice, the $Ra = 10^6$ case was harder to run to completion on the cluster.

(c) $Ra = 10^4$: mesh refinement vs Blankenbach

Blankenbach benchmark 1 reports $Nu \approx 4.884$ for this configuration (cite / quote as in course notes).

N	Nu (your run)	$ Nu - 4.884 $	Notes
16	1.0009	3.8831	
32	0.9962	3.8878	
64	0.9968	3.8872	
128	0.9957	3.8883	

The $Ra = 10^4$ results shown here remain close to $Nu \approx 1$ and do not approach the Blankenbach benchmark $Nu \approx 4.884$. This suggests the benchmark convective state is not being reproduced in these runs. Deviations from 4.884 in our results are most plausibly attributable to numerical/implementation details rather than a different physical regime: in particular, whether the diagnostic flux is being evaluated on the intended boundary (facet/marker conventions), and the sensitivity of the coupled solve to timestep control/solver tolerances and run length. These effects can prevent the computed $Nu(t)$ from cleanly approaching the Blankenbach steady-state benchmark even when Ra and the boundary-value problem match Case 1a.

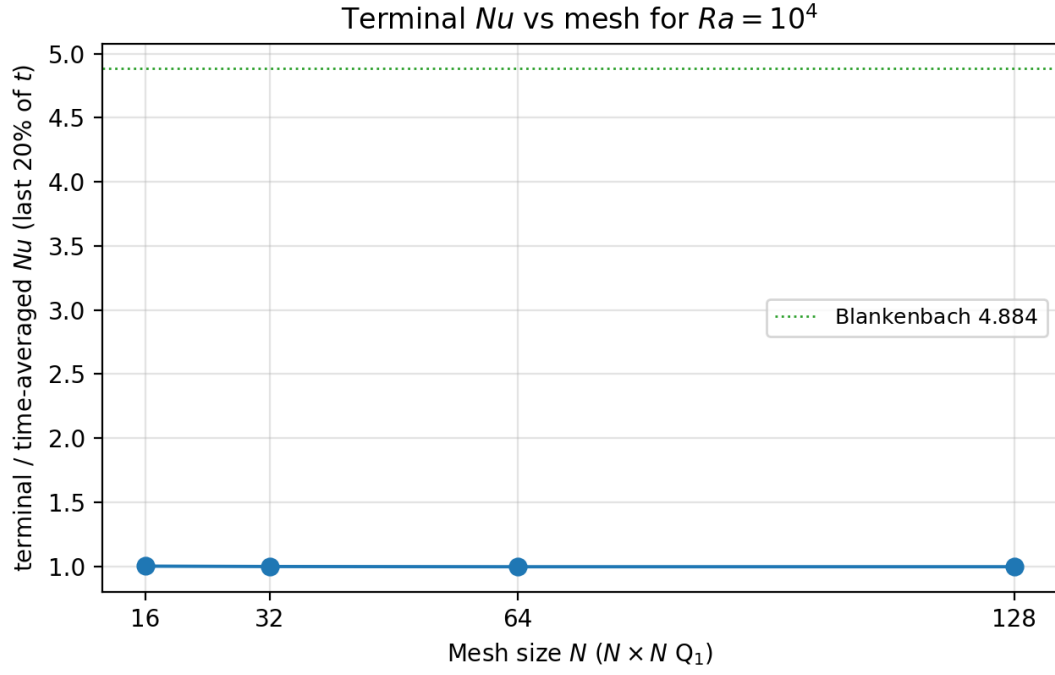


Figure 4: Terminal / time-window averaged Nu vs N for $Ra = 10^4$. Horizontal dotted line: Blankenbach $Nu \approx 4.884$.

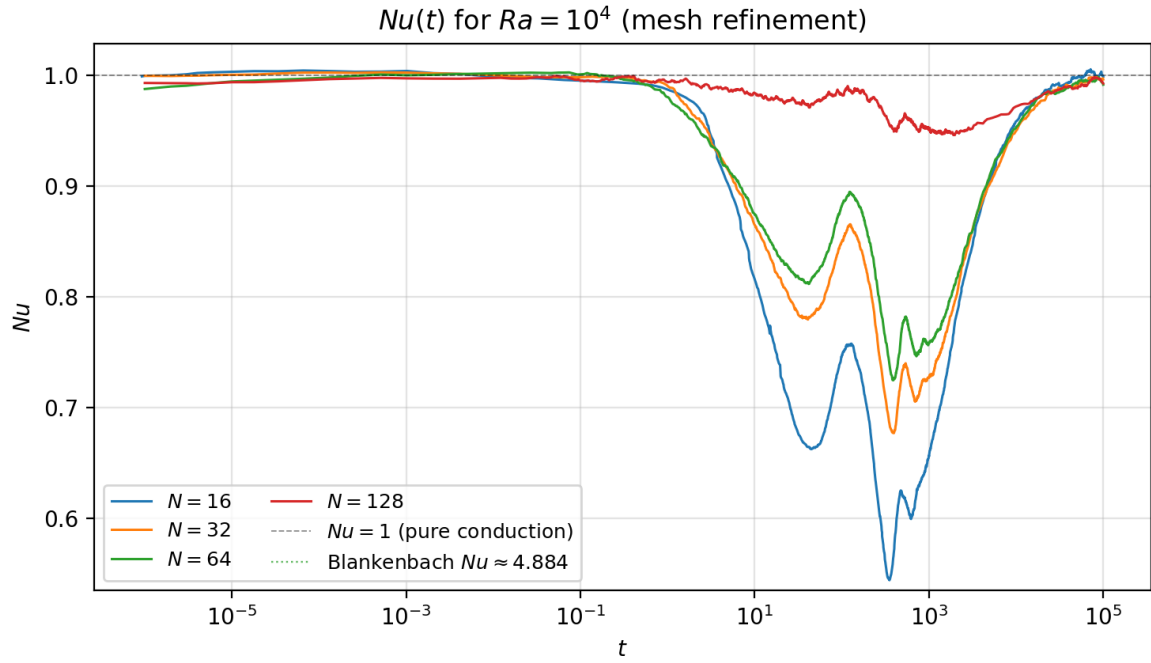


Figure 5: $Nu(t)$ for $Ra = 10^4$ with $N \in \{16, 32, 64, 128\}$ (log-scaled time axis).

Context for the benchmark ≈ 4.884 (Blankenbach Case 1a). A standard way to verify a numerical Rayleigh–Bénard convection implementation is to compare its long-time averaged heat transport against the benchmark suite of Blankenbach et al. (1989). Their *Case 1a* considers a 2D unit aspect-ratio box, incompressible Boussinesq flow with constant viscosity, isothermal top and bottom boundaries, insulating sidewalls, and nondimensional Rayleigh number $Ra = 10^4$. In this setting the Nusselt number

$$Nu(t) = \frac{\text{total vertical heat flux through the boundary}}{\text{purely conductive flux}}$$

quantifies how much convection enhances heat transport beyond conduction (for which $\equiv 1$). For Case 1a, independent codes agree on a steady convective state whose *mean* Nusselt number at the top boundary is approximately ≈ 4.884 . In part (c) we therefore refine the mesh ($N = 16, 32, 64, 128$ at fixed $Ra = 10^4$) and examine whether (t) settles to a mesh-independent plateau and whether the late-time mean approaches the benchmark value. Deviations from 4.884 can indicate that the run has not reached steady state (insufficient $t_{\max}$), that time integration tolerances/time steps are too loose or overly stiff, or that boundary marking/flux evaluation in the discrete implementation is incorrect.